Review the assignment due date

Final Project

Team Number: 4

Team Name: Whear

Team Member Name	Email Address
jefferson ding	tyding@seas.upenn.edu
dimitris deliakidis	ddelias@seas.upenn.edu
carly googel	cagoogel@seas.upenn.edu

GitHub Repository URL: https://github.com/upenn-embedded/final-project-whear

GitHub Pages Website URL: [for final submission]*

Final Project Proposal

1. Abstract

Whear is a smart closet inventory system that tracks which garments are present in your wardrobe in real time. Washable UHF RFID laundry tags are sewn or stuck onto each item, and a YRM100 (Impinj R2000) reader sweeps the closet over a 5–6 dBi UHF patch antenna with a 3–6 m range. The reader is driven by the network core of an nRF5340 (bare-metal C) over UART, which forwards detected EPCs to the application core (Zephyr RTOS) through a shared-memory IPC mailbox. The application core brings up Wi-Fi via the nRF7002 companion chip and streams tag reads as JSON over a WebSocket to a live mobile / web dashboard. A local OLED / TFT display, WS2812B status ring, and piezo buzzer provide at-a-glance feedback without a phone.

2. Motivation

Getting ready shouldn't be stressful, and yet most people routinely can't find the garment they want, don't know whether an item is in the closet, the laundry, or simply misplaced, forget about clothes they haven't worn in weeks, and end up buying duplicates of pieces they already own. Whear fixes this by making the closet itself aware of its contents: every garment is tagged, the reader continuously reconciles presence/absence, and the app surfaces pieces that have gone missing or haven't been worn in a long time. The problem is a good fit for an embedded system — it needs always-on low-power scanning, a real UHF RF front end, strict timing for the RFID framing protocol, and a networked bridge to a phone — so it exercises both the bare-metal and RTOS sides of the course.

3. System Block Diagram

```
┌──────────────────┐   ┌──────────────────────────────────────┐
│  RFID tags on    │ UHF │ UHF patch antenna (5–6 dBi, SMA)     │
│  each garment    │◄────►│                                      │
│ (96-bit EPC)     │   └──────────────────────────────────────┘
└──────────────────┘                        │
                                            ▼
                              ┌──────────────────────┐
                              │  YRM100 RFID reader  │
                              │    (Impinj R2000)    │
                              └──────────────────────┘
                                        │ UART @ 115200
                                        │ (P1.04 TX / P1.05 RX)
                                        ▼
┌──────────────────────────────────────────────────────────────┐
│                      nRF5340 (dual-core)                     │
│                                                              │
│  ┌──────────────────────┐   shared RAM   ┌──────────────────────┐ │
│  │ Network core         │  0x2100FC00    │ Application core     │ │
│  │ (bare-metal C)       │  ring buffer,  │ (Zephyr RTOS)        │ │
│  │                      │  IPC HW signal │                      │ │
│  │ • YRM100 driver      │◄──────────────►│ • Wi-Fi manager      │ │
│  │ • inventory poll     │                │ • WebSocket client   │ │
│  │ • dedup / RSSI       │                │ • IPC consumer       │ │
│  │ • WS2812B / TFT      │                │ • SPU config         │ │
│  └──────────────────────┘                └──────────────────────┘ │
└──────────────────────────────────────────────────────────────┘
                                                   │ QSPI
                                                   ▼
                                        ┌──────────────────────┐
                                        │   nRF7002 Wi-Fi radio │
                                        └──────────────────────┘
                                                   │ 802.11 b/g/n
                                                   ▼
                                        ┌──────────────────────┐
                                        │ WebSocket server     │
                                        │ → mobile / web dashboard │
                                        └──────────────────────┘
```

Local user-facing peripherals driven from the network core: HX8357D TFT display (inventory count + status), WS2812B NeoPixel ring (scan feedback), piezo buzzer, tactile buttons (mode / scan).

4. Design Sketches

See `docs/` and the project presentation for hand sketches of (a) a closet rail with the Whear reader device mounted on one end and tagged garments hanging from it, and (b) the reader box with external SMA antenna and USB power.

5. Software Requirements Specification (SRS)

5.1 Definitions, Abbreviations

- **EPC** — Electronic Product Code, the 96-bit unique ID broadcast by each UHF RFID tag.
- **RSSI** — Received Signal Strength Indicator, reported in dBm by the reader.
- **IPC** — Inter-Processor Communication (nRF5340 hardware peripheral + shared RAM mailbox).
- **SPU** — System Protection Unit (nRF5340 resource permission controller).
- **Net core / App core** — the two Arm Cortex-M33 cores on the nRF5340.
- **Inventory** — one RFID sweep that reports all tags currently in the antenna's field.

5.2 Functionality

ID	Description
SRS-01	The system shall run a continuous multi-inventory sweep on the YRM100 reader and report each detected tag (EPC + RSSI) to the application core within 100 ms of the reader notification.
SRS-02	The system shall de-duplicate tags within a single session and only emit an IPC event the first time a given EPC is seen (up to at least 100 unique tags tracked on the network core).
SRS-03	The application core shall connect to a configured Wi-Fi SSID and obtain a DHCPv4 lease within 45 s of boot, and shall automatically reconnect on disconnect.
SRS-04	The application core shall open a WebSocket connection to the configured server and transmit each new tag as a JSON frame of the form `{"epc":"<hex>","rssi":<int>}` within 1 s of the IPC signal.
SRS-05	A full-ring (8-slot) IPC buffer shall never be overwritten; if the app core falls behind, the net core drops new tags rather than corrupting in-flight entries, and the `total_tags` counter still reflects truth.
SRS-06	Low-power / standby mode shall reduce active sweeping to at most once per 60 s and dim any attached display.

6. Hardware Requirements Specification (HRS)

6.1 Definitions, Abbreviations

- **UHF RFID** — Ultra-High-Frequency (902–928 MHz in US region) passive RFID.
- **YRM100** — UART-based UHF RFID reader module built around the Impinj R2000.
- **nRF7002** — Nordic Wi-Fi 6 companion IC, attached to the nRF5340 over QSPI.
- **LDO** — Low-Drop-Out linear regulator (AMS1117-3.3).

6.2 Functionality

ID	Description
HRS-01	A UHF RFID reader shall be used to detect passive tags on garments at a range of at least 1.5 m (target 3–6 m) through fabric, using a ≥5 dBi patch antenna.
HRS-02	The reader shall be configured for the US region (902.25–927.75 MHz) at ≥23 dBm TX power.
HRS-03	A Wi-Fi radio shall provide 2.4 GHz 802.11 b/g/n connectivity for uplink to the server (nRF7002 over QSPI).
HRS-04	A visual indicator (TFT display and/or NeoPixel ring) shall show current garment count and scan activity without requiring the phone app.
HRS-05	The device shall run from a single 5 V USB supply; the MCU rail shall be provided by a 3.3 V LDO capable of the nRF7002's peak Wi-Fi TX current.
HRS-06	A piezo buzzer shall provide audible feedback (≥55 dB at 10 cm) on successful first-scan of a new tag.

7. Bill of Materials (BOM)

Part	Qty	Purpose
Nordic nRF7002-DK (nRF5340 + nRF7002)	1	Dual-core MCU + Wi-Fi 6 radio
YRM100 UHF RFID reader module	1	Impinj R2000, UART control
UHF patch antenna, ≥5 dBi, SMA	1	Tag interrogation, 3–6 m range
Washable UHF RFID laundry tags	10+	One per tracked garment
HX8357D 3.5" TFT (or SSD1306 OLED)	1	Local inventory / status display
WS2812B NeoPixel ring (8-pixel)	1	Scan / status feedback
Piezo buzzer	1	Audible scan confirmation
Tactile buttons	2	Mode / manual scan
AMS1117-3.3 LDO	1	5 V → 3.3 V rail
5 V USB power supply	1	Main power

8. Final Demo Goals

- Live closet demo: tagged garments placed on / removed from a rack, inventory count on the local display and phone dashboard updating in real time.
- Show at least 10 distinct tags being tracked simultaneously with correct presence/absence transitions.
- Show the “forget-me-not” view surfacing a garment that hasn’t been seen for an extended window.
- Pull the network cable / drop Wi-Fi and show automatic reconnect + no lost already-buffered tags.

9. Sprint Planning

Milestone	Functionality Achieved	Distribution of Work
Sprint #1	YRM100 driver on bare-metal net core; tags detected and logged over RTT	Dimitris: YRM100 driver / net-core bring-up · Jefferson: Zephyr app-core skeleton · Carly: enclosure + antenna mount + BOM
Sprint #2	IPC mailbox between net & app cores; Wi-Fi association + DHCP on nRF7002; WebSocket client sending JSON	Dimitris: IPC producer + dedup · Jefferson: Wi-Fi manager + WebSocket client · Carly: display + NeoPixel libraries
MVP Demo	End-to-end: tag → net core → IPC → app core → WebSocket → dashboard, with on-device display of count	all
Final Demo	Full feature set: laundry tracking, forget-me-not alerts, reconnect logic, low-power mode, polished enclosure	all

This is the end of the Project Proposal section. The remaining sections will be filled out based on the milestone schedule.

Repository Layout

```
final-project-whear/
├── Makefile            # top-level dual-core build (net + app) + flash targets
├── app_core/           # Zephyr application (Cortex-M33 app core)
│   ├── CMakeLists.txt
│   ├── prj.conf        # Wi-Fi, networking, WebSocket, heap, stacks
│   ├── boards/         # DT overlay reserving shared-mem region
│   └── src/
│       ├── main.c          # SPU setup, release net core, main loop
│       ├── ipc_app.{c,h}   # IPC consumer (ring-buffer reader, IRQ)
│       ├── wifi_manager.{c,h}  # nRF7002 association + DHCP
│       └── ws_client.{c,h}     # WebSocket client, JSON-per-tag
├── net_core/           # Bare-metal firmware (Cortex-M33 network core)
│   ├── main.c              # YRM100 init + continuous inventory
│   ├── ipc_net.{c,h}       # IPC producer (writes ring, fires SEND task)
│   ├── nrf5340_net.{h,ld}  # register map + linker script
│   ├── startup_nrf5340_net.c
│   └── lib/
│       ├── yrm100/     # YRM100 RFID driver + UART backend
│       ├── hx8357d/    # TFT display (stub)
│       └── ws2812b/    # NeoPixel ring  (stub)
└── shared/
    └── ipc_shared.h    # mailbox layout used by both cores
```

Build & Flash

Requires Nordic Connect SDK v3.0.2 at `/opt/nordic/ncs/v3.0.2`, the GNU Arm Embedded Toolchain, and `nrfjprog`.

```
make                # build both cores
make flash          # erase + flash net and app cores
make monitor        # open serial console @ 115200
make clean          # clean all build artifacts
```

Individual targets: `make net`, `make app`, `make flash-net`, `make flash-app`, `make erase`, `make recover`.

Wi-Fi credentials and the WebSocket server address are compile-time constants:

- SSID / PSK → `app_core/prj.conf` (`CONFIG_WIFI_CREDENTIALS_STATIC_SSID`, `..._PASSWORD`)
- Server address / port / path → `WS_SERVER_ADDR`, `WS_SERVER_PORT`, `WS_URL_PATH` in [app_core/src/main.c]

Sprint Review #1

Last week's progress

Our first sprint was all about de-risking the riskiest part of the project: whether we could actually read passive UHF RFID tags through fabric at the range we promised in the proposal. We sourced the full core RFID stack — the YRM100 (Impinj R2000) module, a 6 dBi UHF patch antenna with an SMA pigtail, and a handful of washable laundry tags — and got the reader talking to our MCU over UART. We wrote the first

cut of the YRM100 driver from scratch against the manufacturer's frame format (0xBB header, 0x7E terminator, command / response / notice frames with checksum), wired up `start_multi_inventory`, and got continuous tag notifications streaming back with EPC and RSSI. We also stood up a simple dedup table on the MCU so the same tag read twice in a row only fires a "new tag" event once.

Current state of project

Sourced components for the core RFID system. Have the core RFID system functioning end-to-end: multi-inventory runs continuously, new tags are parsed, de-duplicated, and logged, and we can see clean EPC + RSSI on the serial console. We demoed our key systems working during lab section — a tagged garment held up to the antenna reliably reports its EPC, and walking it out of range cleanly stops the "seen" events.

Demo of our device: https://drive.google.com/file/d/1t3F-Fl3bGAu_fCwf2g--YzYmbsHpy9S0/view?usp=sharing

Next week's plan

- Build the enclosure (house the reader board, antenna mount, USB power).
- Get Wi-Fi connected and working — this is the other big unknown, since the ATmega route in the original proposal would have bottlenecked on an ESP32 AT-bridge. We're evaluating moving the whole stack to the nRF7002-DK (nRF5340 + nRF7002) so Wi-Fi is first-class and we get a second core for free.
- Source parts for the LCD screen and wire up the display. https://docs.google.com/spreadsheets/d/1UGfKaoHQOmJi5FX-KRRJ3rrU7xW-E2bn/edit?usp=sharing&ouid=114478550732335716761&rtpof=true&sd=true
- Deploy RFID sensor data to a web endpoint (pick protocol: REST vs. WebSocket).
- Start on mobile interface design and software development.

Sprint Review #2

We have a ios app!! Check out the repo: https://github.com/carlygoogel/Whear.git We have also designed and 3d printed our enclosure, there is some improvements that need to be made but its good enough for MVP.

Last week's progress

Big platform change this sprint: we moved off the ATmega328PB + ESP32 split and onto the nRF7002-DK. The nRF5340 gives us two Cortex-M33s on the same chip, and the nRF7002 gives us a real Wi-Fi 6 radio over QSPI instead of an AT-command bridge, which removes a whole class of latency and buffering problems from the uplink path. We then split the firmware to match the hardware:

- **Network core (bare-metal C):** owns the YRM100 over UARTE0 (P1.04/P1.05), runs continuous multi-inventory, de-duplicates EPCs, and writes new tags into a shared-memory ring buffer. Linker script, startup, and register map are hand-rolled — no Zephyr on this core. See net_core/main.c and net_core/ipc_net.c.
- **Application core (Zephyr RTOS):** configures the SPU to grant the net core access to the RFID UART pins, releases the net core from forced-off, brings up Wi-Fi via the Zephyr `net_mgmt` API with static credentials, waits on a DHCPv4 lease, then opens a WebSocket to the dashboard. See app_core/src/main.c and app_core/src/wifi_manager.c.
- **IPC:** a 192-byte mailbox pinned at `0x2100FC00` in the top 1 KB of net core RAM, carrying a magic word, producer/consumer indices, net status, total tag count, and an 8-slot ring of 20-byte tag entries. The net core writes, issues a `__DMB()`, bumps `write_idx`, and fires the nRF5340 IPC `TASKS_SEND[0]`; the app core's ISR gives a semaphore and the main loop drains the ring. See shared/ipc_shared.h.

On the uplink side, the WebSocket client formats each new tag as `{"epc":"<hex>","rssi":<int>}` and sends it as a masked text frame. We also added reconnect logic on both the Wi-Fi and WebSocket layers so a dropped AP doesn't brick the pipeline.

We put together a top-level `Makefile` that builds both cores, flashes each coprocessor separately with `nrfjprog` (`--coprocessor CP_NETWORK` and `CP_APPLICATION`), and opens a serial monitor — so the whole thing is `make flash` from a clean checkout.

Current state of project

End-to-end tag flow is alive: tagged garment → antenna → YRM100 → UART → net core driver → IPC ring → app core → WebSocket → dashboard. The enclosure has been designed and 3D-printed — a closet-rail-mounted box with an integrated hook that hangs directly on the rod, a window cutout for the TFT, and internal standoffs for the DK and the antenna:

Cad view:
https://cad.onshape.com/documents/d4b02aa7ef074eb8d4de5ae9/w/d47a6f38ca1e84162dad2f7c/e/438206de66ef1abd3ba63d5b?renderMode=0&uiState=69e29febd452e2b512072576

The antenna mounts externally, and the board runs off a single 5 V USB supply. The HX8357D TFT and WS2812B NeoPixel drivers are scaffolded in net_core/lib/ but not yet wired into the main loop — those are the next local-UX piece.

Next week's plan

- Finish the HX8357D driver so the device shows "present / total" counts without needing the phone.
- Wire up the WS2812B ring for scan-activity feedback and the piezo buzzer for first-scan chirps.
- Build the mobile / web dashboard against the live WebSocket stream (presence toggles, last-seen timestamps, forget-me-not list).
- Add a persistence layer on the server so "last seen" survives reader reboots.
- Start measuring: effective read range through fabric, wall-clock latency from tag-waved to dashboard-update, and current draw in idle vs. active scan.

MVP Demo

Final Report

Don't forget to make the GitHub pages public website! If you've never made a GitHub pages website before, you can follow this webpage (though, substitute your final project repository for the GitHub username one in the quickstart guide): https://docs.github.com/en/pages/quickstart

1. Video

2. Images

3. Results

3.1 Software Requirements Specification (SRS) Results

ID	Description	Validation Outcome
SRS-01	YRM100 sweep → IPC within 100 ms.	
SRS-02	De-duplication of EPCs on the network core (≥100 unique tags).	
SRS-03	Wi-Fi associate + DHCP within 45 s, auto-reconnect.	
SRS-04	Each new tag emitted as JSON over WebSocket within 1 s.	
SRS-05	Full IPC ring never corrupts in-flight entries.	
SRS-06	Low-power mode: one sweep per 60 s, dimmed display.	

3.2 Hardware Requirements Specification (HRS) Results

ID	Description	Validation Outcome
HRS-01	UHF RFID read range ≥1.5 m (target 3–6 m) through fabric with ≥5 dBi antenna.	
HRS-02	Reader configured for US region at ≥23 dBm.	
HRS-03	2.4 GHz Wi-Fi b/g/n uplink via nRF7002.	
HRS-04	Local display + NeoPixel ring reflect inventory / scan activity.	
HRS-05	Single 5 V USB supply → 3.3 V LDO sustains peak Wi-Fi TX.	
HRS-06	Piezo buzzer ≥55 dB @ 10 cm on first-scan of a new tag.	

4. Conclusion

References